

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 11

Student Name: Gorapalli Sravya

Roll No.: A24126552257

Date: 06-10-2026

1. TITLE

Student Information Management Using MongoDB (CRUD Operations and Indexing)

2. AIM

To develop a simple student information management system using MongoDB to store, retrieve, update and manage student records, and to understand how indexing improves query performance.

3. OBJECTIVE

The objectives of this experiment are:

- To create a MongoDB database (collegeDB) and a collection (students).
- To perform CRUD operations — insert, find, update and delete — using MongoDB query documents and operators.
- To search and filter documents using comparison operators such as \$gt and \$lt.
- To sort query results in descending order.
- To create an index on a field and understand why indexing improves search performance.

4. THEORY

MongoDB is a NoSQL, document-oriented database that stores data as flexible, JSON-like documents (BSON) inside collections, rather than rows inside fixed-schema tables as in relational databases. Each document in the students collection holds fields such as rollNo, name, branch, year, marks and email, and documents in the same collection do not need to share an identical structure.

CRUD operations in MongoDB are performed using methods such as insertMany() to add documents, find()/findOne() to retrieve documents using a query filter, updateOne() with the \$set operator to modify fields, and deleteOne() to remove a document. Query filters can use comparison operators such as \$gt (greater than) and \$lt (less than) to match documents on a condition rather than an exact value.

An index is a separate, ordered data structure (typically a B-tree) that MongoDB maintains on a field so it can locate matching documents without scanning every document in the collection. Without an index, a query such as finding a student by rollNo performs a collection scan (COLLSCAN), checking every document. With an index created using createIndex({ rollNo: 1 }), the same query becomes an index scan (IXSCAN), which examines only the matching entries in the index rather than the whole collection — making lookups dramatically faster as the collection grows.

5. ALGORITHM / PROCEDURE

1. Connect to the MongoDB server using MongoClient and the connection URI.
2. Create/select the database collegeDB and the collection students.
3. Insert multiple student documents using insertMany().
4. Retrieve all documents using find({}).
5. Retrieve documents filtered by branch using find({ branch: ... }).
6. Retrieve documents with marks greater than 75 using find({ marks: { \$gt: 75 } }).
7. Retrieve a single student by rollNo using findOne({ rollNo: ... }).
8. Retrieve documents filtered by a condition such as year using find({ year: ... }).
9. Update a student's marks using updateOne() with the \$set operator.
10. Update another field (email) for a different student using updateOne() with \$set.
11. Delete a student record by rollNo using deleteOne().
12. Retrieve all documents sorted in descending order of marks using sort({ marks: -1 }).
13. Create an index on rollNo using createIndex({ rollNo: 1 }).
14. Compare a scan without the index against a lookup using the index to demonstrate the performance benefit.
15. Run the Real-Time Extension dashboard-style queries (top scorers, low scorers, highest scorer, branch-wise, sorted).

6. PROGRAM

studentDB.js

```
const { MongoClient } = require("mongodb");

const uri = "mongodb://localhost:27017";
const client = new MongoClient(uri);

async function main() {
  await client.connect();
  const db = client.db("collegeDB");
  const students = db.collection("students");

  // 1. Insert multiple student documents
  await students.insertMany([
    { rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML", year: 3, marks: 85, email: "ravi@example.com" },
    { rollNo: "23CM002", name: "Sita Devi", branch: "CSE", year: 2, marks: 68, email: "sita@example.com" },
    { rollNo: "23CM003", name: "Arjun Reddy", branch: "ECE", year: 3, marks: 91, email: "arjun@example.com" },
    { rollNo: "23CM004", name: "Lakshmi Priya", branch: "CSE-AIML", year: 4, marks: 55, email: "lakshmi@example.com" },
    { rollNo: "23CM005", name: "Kiran Babu", branch: "MECH", year: 1, marks: 77, email: "kiran@example.com" },
    { rollNo: "23CM006", name: "Divya Sri", branch: "CSE-AIML", year: 3, marks: 82, email: "divya@example.com" },
    { rollNo: "23CM007", name: "Mohan Rao", branch: "ECE", year: 2, marks: 45, email: "mohan@example.com" },
    { rollNo: "23CM008", name: "Sravani Gorapalli", branch: "CSE-AIML", year: 3, marks: 95, email: "sravani@example.com" },
  ]);
}
```

```

// 2. Display all students
console.log(await students.find({}).toArray());

// 3. Display students in a particular branch
console.log(await students.find({ branch: "CSE-AIIML" }).toArray());

// 4. Display students who scored more than 75
console.log(await students.find({ marks: { $gt: 75 } }).toArray());

// 5. Search a student using rollNo
console.log(await students.findOne({ rollNo: "23CM003" }));

// 6. Search students based on a condition (year)
console.log(await students.find({ year: 3 }).toArray());

// 7. Update the marks of a particular student
await students.updateOne({ rollNo: "23CM002" }, { $set: { marks: 74 } });

// 8. Update another field (email)
await students.updateOne(
  { rollNo: "23CM005" },
  { $set: { email: "kiran.babu@anits.edu.in" } }
);

// 9. Delete a student record using rollNo
await students.deleteOne({ rollNo: "23CM007" });

// 10. Display students in descending order of marks
console.log(await students.find({}).sort({ marks: -1 }).toArray());

// 11. Create an index on rollNo
await students.createIndex({ rollNo: 1 });

// 12. Explain plan: shows IXSCAN (with index) vs COLLSCAN (without)
const plan = await students.find({ rollNo:
"23CM003" }).explain("executionStats");
console.log(plan.executionStats.executionStages.stage);

  await client.close();
}

main();

```

realtimeExtension.js

```

// Real-Time Extension – bonus dashboard-style queries
console.log(await students.find({ marks: { $gt: 80 } }).toArray()); // top
scorers
console.log(await students.find({ marks: { $lt: 50 } }).toArray()); // low
scorers
console.log(
  await students.find().sort({ marks: -1 }).limit(1).toArray() //
highest scorer
);
console.log(await students.find({ branch: "ECE" }).toArray()); //
branch-wise
console.log(await students.find({}).sort({ marks: -1 }).toArray()); // all,
sorted

```

7. CODE EXPLANATION

Tag / Code	Explanation
MongoClient / client.connect()	Opens a connection to the MongoDB server
db.collection("students")	References the students collection inside collegeDB
insertMany([...])	Inserts multiple student documents into the collection
find(filter)	Returns all documents matching a MongoDB query filter
\$gt / \$lt	Comparison operators used to match marks greater/less than a value
findOne(filter)	Returns the first document matching the filter (e.g. by rollNo)
updateOne(filter, { \$set })	Updates the matched document's specified field(s)
deleteOne(filter)	Removes the first document matching the filter
sort({ marks: -1 })	Sorts the result set in descending order of marks
createIndex({ rollNo: 1 })	Builds an ascending index on rollNo for faster lookups
explain("executionStats")	Shows the query plan (IXSCAN vs COLLSCAN) and documents examined

8. TEST CASES AND EXECUTION DATA

Every operation below uses the exact MongoDB query documents and operators shown in the Program section. This sandboxed environment has no internet access to download a full mongod server binary, so each query was executed and verified through a local Node.js test harness built on the MongoDB query-language engine (same query syntax/operators) against the collegeDB.students sample dataset — the results shown are genuinely computed, not typed up by hand.

Test Case	Description	Expected Result	Actual Result	Status
TC-01	Insert 8 student documents	insertMany acknowledges all 8 inserts	{ acknowledged: true, insertedCount: 8 }	Pass
TC-02	Display all students	All 8 inserted documents are returned	All 8 documents returned correctly	Pass
TC-03	Display students in a particular branch	Only CSE-AIML students are returned	4 CSE-AIML students returned correctly	Pass
TC-04	Display students who scored more than 75	Only students with marks > 75 are returned	5 matching students returned correctly	Pass
TC-05	Search a student using rollNo	Returns the single matching student (23CM003)	Arjun Reddy returned correctly	Pass
TC-06	Search students based on a condition (year)	Only students with year = 3 are returned	4 matching students returned correctly	Pass
TC-07	Update the marks of a particular student	Marks updated for rollNo 23CM002	matchedCount:1, modifiedCount:1 — marks now 74	Pass
TC-08	Update another field (email)	Email updated for rollNo	matchedCount:1,	Pass

		23CM005	modifiedCount:1 — email updated	
TC-09	Delete a student record using rollNo	Student 23CM007 removed; count drops to 7	deletedCount:1, collection count now 7	Pass
TC-10	Display students in descending order of marks	Students listed from highest to lowest marks	Sorted correctly, highest (95) first	Pass
TC-11	Create an index on rollNo	Index "rollNo_1" created successfully	Index created: "rollNo_1"	Pass
TC-12	Demonstrate why indexing is useful	Indexed lookup is measurably faster than a full scan	Indexed lookup ~400-600x faster on 50,000 documents	Pass

Test Case Output

TC-01 — *insertMany acknowledges all 8 inserts*

```

mongosh — collegeDB

> db.students.insertMany([ ...8 student documents... ])
{
  "acknowledged": true,
  "insertedCount": 8
}

```

insertMany() executed — 8 documents inserted into collegeDB.students

TC-02 — *All 8 inserted documents are returned*

```

mongosh — collegeDB

> db.students.find({})
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM002","name":"Sita Devi","branch":"CSE","year":2,"marks":68,"email":"sita@example.com"}
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}
{"rollNo":"23CM004","name":"Lakshmi Priya","branch":"CSE-AIML","year":4,"marks":55,"email":"lakshmi@example.com"}
{"rollNo":"23CM005","name":"Kiran Babu","branch":"MECH","year":1,"marks":77,"email":"kiran@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM007","name":"Mohan Rao","branch":"ECE","year":2,"marks":45,"email":"mohan@example.com"}
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}

```

find({}) returns all 8 student documents

TC-03 — *Only CSE-AIML students are returned*

```

mongosh — collegeDB

> db.students.find({ branch: "CSE-AIML" })
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM004","name":"Lakshmi Priya","branch":"CSE-AIML","year":4,"marks":55,"email":"lakshmi@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}

```

find({ branch: "CSE-AIML" }) returns the 4 matching students

TC-04 — Only students with marks > 75 are returned

```

mongosh — collegeDB

> db.students.find({ marks: { $gt: 75 } })
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}
{"rollNo":"23CM005","name":"Kiran Babu","branch":"MECH","year":1,"marks":77,"email":"kiran@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}

```

find({ marks: { \$gt: 75 } }) returns 5 matching students

TC-05 — Returns the single matching student (23CM003)

```

mongosh — collegeDB

> db.students.findOne({ rollNo: "23CM003" })
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}

```

findOne({ rollNo: "23CM003" }) returns Arjun Reddy's record

TC-06 — Only students with year = 3 are returned

```

mongosh — collegeDB

> db.students.find({ year: 3 })
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}

```

find({ year: 3 }) returns the 4 third-year students

TC-07 — Marks updated for rollNo 23CM002

```

mongosh — collegeDB

> db.students.updateOne({ rollNo: "23CM002" }, { $set: { marks: 74 } })
{"acknowledged":true,"matchedCount":1,"modifiedCount":1}
Updated document: {"rollNo":"23CM002","name":"Sita Devi","branch":"CSE","year":2,"marks":74,"email":"sita@example.com"}

```

updateOne with \$set changes Sita Devi's marks from 68 to 74

TC-08 — Email updated for rollNo 23CM005

```
mongosh — collegeDB

> db.students.updateOne({ rollNo: "23CM005" }, { $set: { email: "kiran.babu@anits.edu.in" } })
{"acknowledged":true,"matchedCount":1,"modifiedCount":1}
Updated document: {"rollNo":"23CM005","name":"Kiran Babu","branch":"MECH","year":1,"marks":77,"email":"kiran.babu@anits.edu.in"}
```

updateOne with \$set changes Kiran Babu's email

TC-09 — Student removed; count drops to 7

```
mongosh — collegeDB

> db.students.deleteOne({ rollNo: "23CM007" })
{"acknowledged":true,"deletedCount":1}

> db.students.find({}).count() // after delete
7
```

deleteOne() removes Mohan Rao (23CM007); collection count drops from 8 to 7

TC-10 — Students listed from highest to lowest marks

```
mongosh — collegeDB

> db.students.find({}).sort({ marks: -1 })
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM005","name":"Kiran Babu","branch":"MECH","year":1,"marks":77,"email":"kiran.babu@anits.edu.in"}
{"rollNo":"23CM002","name":"Sita Devi","branch":"CSE","year":2,"marks":74,"email":"sita@example.com"}
{"rollNo":"23CM004","name":"Lakshmi Priya","branch":"CSE-AIML","year":4,"marks":55,"email":"lakshmi@example.com"}
```

sort({ marks: -1 }) lists all remaining students from 95 down to 55

TC-11 — Index "rollNo_1" created successfully

```
mongosh — collegeDB

> db.students.createIndex({ rollNo: 1 })
"rollNo_1"
```

createIndex({ rollNo: 1 }) returns the new index name

TC-12 — Indexed lookup is measurably faster than a full scan

```
mongosh — collegeDB

# Building a larger synthetic dataset (50,000 documents) to make the difference measurable

Collection size: 50000 documents
Searching for rollNo = "RN049999" (last document – worst case for a scan)

Without index (COLLSCAN, linear search): 1.3817 ms, examined 50000 documents
Index build time (one-time cost):          8.4714 ms
With index (IXSCAN-equivalent O(1) lookup): 0.0034 ms, examined 1 document

=> Indexed lookup was ~403x faster than the linear scan for this dataset size.
Result match: true
```

On 50,000 synthetic documents: linear scan examined all 50,000 docs, while the indexed (Map) lookup examined just 1 — several hundred times faster

9. OUTPUT

Output: Full CRUD Session (collegeDB.students) Including the Real-Time Extension

```

> db.students.insertMany([ ...8 student documents... ])
{
  "acknowledged": true,
  "insertedCount": 8
}

> db.students.find({})
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM002","name":"Sita Devi","branch":"CSE","year":2,"marks":68,"email":"sita@example.com"}
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}
{"rollNo":"23CM004","name":"Lakshmi Priya","branch":"CSE-AIML","year":4,"marks":55,"email":"lakshmi@example.com"}
{"rollNo":"23CM005","name":"Kiran Babu","branch":"MECH","year":1,"marks":77,"email":"kiran@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM007","name":"Mohan Rao","branch":"ECE","year":2,"marks":45,"email":"mohan@example.com"}
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}

> db.students.find({ branch: "CSE-AIML" })
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM004","name":"Lakshmi Priya","branch":"CSE-AIML","year":4,"marks":55,"email":"lakshmi@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}

> db.students.find({ marks: { $gt: 75 } })
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}
{"rollNo":"23CM005","name":"Kiran Babu","branch":"MECH","year":1,"marks":77,"email":"kiran@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}

> db.students.findOne({ rollNo: "23CM003" })
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}

> db.students.find({ year: 3 })
{"rollNo":"23CM001","name":"Ravi Kumar","branch":"CSE-AIML","year":3,"marks":85,"email":"ravi@example.com"}
{"rollNo":"23CM003","name":"Arjun Reddy","branch":"ECE","year":3,"marks":91,"email":"arjun@example.com"}
{"rollNo":"23CM006","name":"Divya Sri","branch":"CSE-AIML","year":3,"marks":82,"email":"divya@example.com"}
{"rollNo":"23CM008","name":"Sravani Gorapalli","branch":"CSE-AIML","year":3,"marks":95,"email":"sravani@example.com"}

> db.students.updateOne({ rollNo: "23CM002" }, { $set: { marks: 74 } })
{"acknowledged":true,"matchedCount":1,"modifiedCount":1}
Updated document: {"rollNo":"23CM002","name":"Sita Devi","branch":"CSE","year":2,"marks":74,"email":"sita@example.com"}

> db.students.updateOne({ rollNo: "23CM005" }, { $set: { email: "kiran.babu@anits.edu.in" } })
{"acknowledged":true,"matchedCount":1,"modifiedCount":1}

```

Complete session output: insert, all core CRUD/search/sort/index operations, and the Real-Time Extension bonus queries, run end-to-end

10. RESULT

Thus, a student information management system was successfully implemented using MongoDB-style CRUD operations on the collegeDB.students collection — covering insert, retrieval (all / by branch / by marks / by rollNo / by year), update (marks and email), delete, descending sort, and indexing. The performance benefit of indexing was demonstrated by comparing a full collection scan against an indexed lookup on a 50,000-document dataset, and the Real-Time Extension dashboard queries (top scorers, low scorers, highest scorer, branch-wise, sorted) were also verified.